Sommario delle funzioni R

Lezioni 1 e 2: riepilogo per la prova intermedia

Paolo Bosetti

Indice

1 Ambiente e pacchetti 1

2 Vettori, matrici, tabelle, liste 2

3 Statistica descrittiva 3

4 Programmazione 3

5 Distribuzioni di probabilità 4

6 Test statistici 5

7 Lettura e scrittura di file 5

8 Tidyverse: manipolazione dati 6
- 8.1 Pipe e tibble . . . 6
- 8.2 `dplyr` . . . 6
- 8.3 `tidyr` . . . 7
- 8.4 `purrr` . . . 7

9 Grafici con `ggplot2` *(tv)* 8

10 Modelli 9
- 10.1 Sintassi delle formule . . . 9
- 10.2 Stima, verifica e previsione . . . 9

11 Errori più frequenti 10

Per ogni funzione: a cosa serve, come si usa e l'errore più frequente. Le funzioni indicate con *(tv)* richiedono `library(tidyverse)`, quelle con *(adas)* richiedono `library(adas.utils)`. Per tutti gli argomenti e i default, consultare `?nome_funzione`.

1 Ambiente e pacchetti

Funzione	Scopo e uso	Attenzione
`install.packages("pkg")`	Installa un pacchetto (una volta sola).	Da eseguire in console, non nel `.qmd` (verrebbe ripetuto a ogni rendering).

Funzione	Scopo e uso	Attenzione
`library(pkg)`	Carica un pacchetto installato.	Va messo nel chunk di setup in testa al documento.
`pkg::fun()`	Usa una funzione senza caricare il pacchetto, es. `outliers::grubbs.test()`.	Il pacchetto deve comunque essere installato.
`source("file.R")`	Esegue uno script e rende disponibili le funzioni che definisce.	Il percorso è relativo alla cartella del `.qmd`.
`knitr::kable(df)`	Formatta una tabella per il documento finale.	Solo per la visualizzazione.

2 Vettori, matrici, tabelle, liste

Funzione	Scopo e uso	Attenzione
`c(...)`	Concatena valori in un vettore. Operazioni aritmetiche e confronti sono elemento per elemento.	Il vettore più corto viene **riciclato**; l'avviso compare solo se le lunghezze non sono multiple. Un vettore ha un solo tipo: `c(1, "a")` diventa testo.
`a:b`, `seq(inizio, fine, by=)`, `seq(inizio, fine, length.out=)`	Sequenza di interi, a passo fisso, o con numero fisso di punti.	`by` e `length.out` sono alternativi. `length(v)-2:0` vale `length(v)-(2:0)`.
`v[i]`, `v[-i]`, `v[v > 5]`	Indicizzazione per posizione, per esclusione, per condizione logica.	Gli indici partono da **1**. Condizioni combinate con `&` (AND) e `|` (OR).
`length(v)`, `length(v) <- n`	Lunghezza; con assegnazione allunga (aggiungendo `NA`) o tronca.	Utile per pareggiare due campioni prima di metterli in una tabella.
`head(x, n)`, `tail(x, n)`	Primi/ultimi `n` elementi o righe (default 6).	
`rep(x, each=)`	Ripete ogni elemento `each` volte.	`rep(1:n(), each=3)` dentro `slice()` replica le righe.
`matrix(v, nrow=, byrow=)`	Crea una matrice.	Per default riempie **per colonne**.
`%*%`, `t()`	Prodotto matriciale, trasposta.	`*` è il prodotto elemento per elemento; `%*%` richiede colonne(A) = righe(B).
`data.frame(...)`	Tabella con colonne eterogenee.	Una colonna non può riferirsi a un'altra appena definita (con `tibble()` sì).
`df$col`, `df[["col"]]`, `df[i, j]`	Colonna come vettore; elemento o sottotabella.	Su una tibble `df[, 2]` restituisce una tibble di una colonna, non un vettore: usare `$` o `pull()`.
`list(a=..., b=...)`, `l$a`, `l[[2]]`	Collezione eterogenea con nomi. I risultati dei test sono liste.	`l[2]` restituisce una lista con un elemento; `l[[2]]` l'elemento stesso.
`attributes(x)`	Mostra i componenti di un oggetto, es. il risultato di un test.	Serve per sapere cosa estrarre con `$`.

Funzione	Scopo e uso	Attenzione
`dim(df)`, `LETTERS`	Numero di righe e colonne; vettore delle lettere maiuscole.	

3 Statistica descrittiva

Funzione	Scopo e uso	Attenzione
`sum`, `mean`, `sd`, `var`, `min`, `max`	Statistiche campionarie.	Un solo `NA` dà `NA`: usare `na.rm=TRUE`. `sd` e `var` dividono per $N-1$.
`range(x)`, `diff(x)`	Minimo e massimo; differenze tra elementi successivi.	`diff(range(x))` è l'ampiezza del campione.
`quantile(x)`	Quantili **campionari** (default 0, 25, 50, 75, 100%).	Da non confondere con `qnorm()`, `qt()`, … che sono quantili **teorici**.
`cov(x, y)`, `cor(x, y)`	Covarianza; correlazione di Pearson (tra -1 e 1).	`cov` dipende dalle unità di misura. `cor` misura solo la dipendenza **lineare**. Su una matrice restituisce la matrice di correlazione.
`is.na(x)`, `na.omit(x)`	Individua i `NA`; li elimina.	`x == NA` dà sempre `NA`. Su una tabella `na.omit` elimina le righe con `NA` in **qualsiasi** colonna.
`which.max(x)`	Indice del massimo (`which.min()` per il minimo).	Restituisce la posizione, non il valore: il valore è `x[which.max(x)]`.
`round(x, n)`	Arrotonda a `n` decimali.	
`nclass.Sturges`	Numero di classi di un istogramma (regola di Sturges).	Si passa senza parentesi: `bins=nclass.Sturges`.

4 Programmazione

Funzione	Scopo e uso	Attenzione
`f <- function(a, b=2) {...}`	Definisce una funzione. Restituisce l'ultimo valore calcolato (o quello di `return()`).	Va assegnata a un nome. Gli argomenti con default sono opzionali.
`\(x) espressione`	Funzione anonima, equivalente a `function(x) espressione`.	Si usa dentro `map_*()` e `geom_function()`.
`for (e in v) {...}`	Ciclo sugli elementi di `v` (o sugli indici `1:length(v)`).	Spesso sostituibile da un'operazione vettoriale, più semplice e veloce.
`if (cond) ... else ...`	Esecuzione condizionale.	`cond` deve essere un **singolo** `TRUE`/`FALSE`: non funziona su una colonna dentro `mutate()`.
`ifelse(test, sì, no)`	Condizionale vettorializzato.	Il risultato è lungo quanto `test`.

Funzione	Scopo e uso	Attenzione
`cut(x, breaks, labels, right=FALSE)`	Suddivide un numerico in classi (vettorializzata).	`labels` ha un elemento in meno di `breaks`. Con `right=FALSE` gli intervalli sono `[a, b)`.
`invisible(x)`	Restituisce `x` senza stamparlo.	Il risultato si vede solo assegnandolo a una variabile.
`cat(...)`, `paste(...)`	Stampa testo; unisce stringhe separandole con uno spazio.	`cat` non va a capo da sola: aggiungere `"\n"`.
`set.seed(n)`	Fissa il seme del generatore casuale.	Senza, i numeri casuali cambiano a ogni esecuzione e i risultati non sono riproducibili.
`sample(n)`, `sample(x, size, replace, prob)`	Permutazione casuale di `1:n`; estrazione casuale da `x`.	Per default **senza** reinserimento: per `size > length(x)` serve `replace=TRUE`. `prob` viene normalizzato.
`factor(x)`, `as.factor(x)`	Converte in variabile categorica (con livelli).	Indispensabile per ANOVA, Tukey e boxplot quando i livelli sono numeri.
`as.numeric(x)`	Converte in numero (`TRUE` $\to$ 1).	Su un fattore restituisce il **codice** del livello, non il valore originale.
`all.equal(a, b)`	Confronta numeri reali con una tolleranza.	Se i valori differiscono non restituisce `FALSE` ma un testo: in un `if` usare `isTRUE(all.equal(a, b))`.
`x -> a`	Assegnazione verso destra, equivale a `a <- x`.	Legale ma meno leggibile.

5 Distribuzioni di probabilità

Il nome è **prefisso + radice**. Radici: `norm`, `unif`, `t`, `chisq`, `f`, `binom`, `pois`, `geom`.

Prefisso	Restituisce	Esempio
`d`	densità $f(x)$ (PDF; probabilità per le discrete)	`dnorm(x, mean=0, sd=1)`
`p`	ripartizione $F(x) = P(X \leq x)$ (CDF)	`pnorm(1.96)`
`q`	quantile, inversa della CDF: $F^{-1}(p)$	`qt(0.975, df=10)`
`r`	`n` numeri casuali	`rnorm(n, mean, sd)`

! Attenzione

- `lower.tail=FALSE`: per `p*()` restituisce l'area a **destra**, $P(X > x)$; per `q*()` interpreta la probabilità come area a destra.
- p-value bilaterale: `2 * pt(abs(t0), df, lower.tail=FALSE)`. Valore critico bilaterale: `qt(1 - alpha/2, df)`.
- `rnorm()` e `dnorm()` vogliono la **deviazione standard**, non la varianza.
- In `dt()`, `pt()`, `qt()` l'argomento `df` indica i gradi di libertà e non ha niente a che fare con una tabella `df`.

6 Test statistici

Regola comune: **p-value < → si rifiuta** H_0; `conf.level` vale $1-\alpha$. Tutti i test restituiscono una lista con `$p.value`, `$statistic`, `$conf.int`, `$estimate`.

Funzione	H_0 e uso	Attenzione
`t.test(x, y)`, `t.test(y ~ g, data=df)`	Medie uguali. Opzioni: `var.equal`, `conf.level`. Con la formula `g` deve avere **due** livelli.	Per default `var.equal=FALSE`, cioè test di **Welch**. Decidere con `var.test()`: `var.equal = vt$p.value > 0.05`.
`t.test(x, mu=m0)`	Un campione: media uguale a `m0`.	Senza `mu`, il confronto è con 0.
`t.test(x, y, paired=TRUE)`	Dati accoppiati: media delle differenze nulla.	`x` e `y` devono avere la stessa lunghezza e lo stesso ordine. Da R 4.4 `paired` non è ammesso con la formula `y ~ g`: usare i vettori oppure `t.test(Pair(x, y) ~ 1, data=df)`.
`var.test(x, y)`, `var.test(y ~ g, data=df)`	Varianze uguali (test F di Fisher).	Presuppone campioni normali.
`shapiro.test(x)`	Il campione è normale: un p-value piccolo indica **non** normalità.	Basta un solo outlier per farlo fallire. Ammette da 3 a 5000 valori. Su un modello va applicato a `residuals(m)`.
`cor.test(x, y)`	Correlazione nulla.	Rileva solo la dipendenza lineare.
`outliers::grubbs.test(x)`	Nessun outlier; H_1: il valore più lontano dalla media è un outlier.	Testa un solo valore. Il pacchetto `outliers` va installato.
`chauvenet(x)` (scritta a lezione)	Criterio, non test: scarta il valore più lontano se $N \cdot P_s < 0.5$.	Si applica **una sola volta**, mai in modo iterativo.

7 Lettura e scrittura di file

I percorsi sono relativi alla cartella del `.qmd`. Le funzioni `read.*()` restituiscono un `data.frame`; le `read_*()` di `readr` *(tv)* restituiscono una `tibble`.

Funzione	Scopo e uso	Attenzione
`read.fwf(file, widths, skip=, col.names=)`	Campi a larghezza fissa.	Serve il vettore delle larghezze in caratteri.
`read.table(file, header=TRUE, sep="", dec=".")`	Testo delimitato, molto configurabile. Accetta anche un URL.	Per default `header=FALSE`: senza `header=TRUE` i nomi delle colonne diventano la prima riga di dati.
`read.delim(file, sep="")`	Variante di `read.table()` con `header=TRUE`.	Il separatore di default è la tabulazione; `sep=""` accetta spazi multipli.
`read.csv()`, `read.csv2()`	CSV anglosassone (`,` e punto decimale); CSV latino (`;` e virgola decimale).	Con la variante sbagliata si ottiene un'unica colonna o numeri letti come testo.

Funzione	Scopo e uso	Attenzione
`read_csv()`, `read_csv2()` *(tv)*	Come sopra, ma più veloci; riportano il tipo di ogni colonna.	Trattino basso al posto del punto.
`write.csv(df, file, row.names=FALSE)`	Salva una tabella.	Senza `row.names=FALSE` aggiunge una colonna di indici. `write_csv()` non la aggiunge mai.
`readLines(file, n)`	Legge le prime `n` righe come testo.	Utile per ispezionare un file prima di importarlo.
`examples_url("file.dat")` *(adas)*	URL completo dei dati di esempio del corso.	Da passare a `read.table(..., header=TRUE)`.

8 Tidyverse: manipolazione dati

8.1 Pipe e tibble

Funzione	Scopo e uso	Attenzione
`%>%` *(tv)*	Passa il valore a sinistra come **primo** argomento della funzione a destra.	Se la tabella va in un altro argomento si usa il segnaposto `.`: `lm(y ~ x, data=.)`. Con la pipe nativa `|>` il segnaposto è `_`, sempre con argomento nominato.
`tibble(...)` *(tv)*	Tabella moderna: ogni colonna può usare quelle definite prima.	Un valore singolo viene ripetuto su tutte le righe; lunghezze diverse da 1 e incompatibili danno errore.

8.2 `dplyr`

All'interno di queste funzioni i nomi delle colonne si scrivono senza `$` e senza virgolette. La tabella originale non viene modificata: per conservare il risultato bisogna assegnarlo.

Funzione	Scopo e uso	Attenzione
`filter(cond, ...)`	Mantiene le righe che soddisfano la condizione; la virgola equivale a AND.	Confronto con `==`, non `=`. Le righe in cui la condizione vale `NA` vengono scartate.
`select(a:c, -d, nuovo=vecchio)`	Sceglie, esclude o rinomina colonne.	
`mutate(nuova = espr)`	Crea o modifica colonne, utilizzabili subito nella stessa chiamata.	La funzione riceve l'intera colonna, quindi deve essere vettorializzata.
`arrange(a, desc(b))`	Ordina le righe (crescente; `desc()` per decrescente).	I `NA` vanno sempre in fondo.

Funzione	Scopo e uso	Attenzione
`slice_head(n=)`, `slice_tail(n=)`, `slice_sample(n=)`, `slice(i)`, `slice_max()`, `slice_min()`	Selezionano righe per posizione, a caso o per valore. In `slice_sample()` si può dare `prop=` al posto di `n=`.	`slice_sample(prop=0.1)` estrae il 10% delle righe.
`group_by(g)`	Prepara le operazioni per gruppo; i dati non cambiano.	Il raggruppamento resta attivo per tutte le operazioni successive.
`summarise(nome = f(col), .by=)`	Riduce ogni gruppo a una riga. `n()` conta le righe del gruppo.	Ogni espressione deve dare **un solo valore**, es. `shapiro.test(x)$p.value`. Dopo `group_by()` su più colonne usare `.groups="drop"`; `.by=` sostituisce `group_by()` e non lascia gruppi.
`rowwise()`	Tratta ogni riga come un gruppo, così si possono usare funzioni non vettorializzate.	È lento e resta attivo fino a `ungroup()`.
`group_split()`	Divide la tabella in una lista di tabelle, una per gruppo.	
`pull(col)`	Estrae una colonna come vettore.	
`first(x)`	Primo elemento.	Utile in `summarise()` per una colonna costante nel gruppo.

8.3 `tidyr`

Funzione	Scopo e uso	Attenzione
`pivot_longer(cols, names_to=, values_to=)`	Da formato largo a lungo (*tidy*). Opzioni: `names_prefix`, `values_drop_na`.	`cols` si può dare come `c1:c3`, `-x`, `!x`, `starts_with("wk")`. Il formato lungo serve per `ggplot` e per le formule.
`pivot_wider(names_from=, values_from=)`	Da formato lungo a largo.	
`expand.grid(x1=..., x2=...)`	Tabella con tutte le combinazioni dei valori (griglia).	Le righe sono il prodotto delle lunghezze: $50 \times 50 = 2500$.

8.4 `purrr`

Funzione	Scopo e uso	Attenzione
`map(.x, .f)`	Applica `.f` a ogni elemento; restituisce una **lista**.	
`map_dbl()`, `map_int()`, `map_chr()`, `map_lgl()`	Come `map()`, ma restituiscono un vettore del tipo indicato.	Errore se un risultato non è un singolo valore di quel tipo.
`map2_*(.x, .y, .f)`	Applica `.f` a due vettori in parallelo.	I due vettori devono avere la stessa lunghezza.
`pmap_*(tabella, .f)`	Numero qualsiasi di argomenti, passati per nome.	Gli argomenti di `.f` devono chiamarsi come le colonne.

Consiglio

Le `map_*()` servono per funzioni **non** vettorializzate (tipicamente quelle con `if`). Quando esiste una versione vettorializzata (`ifelse()`, `cut()`, `paste()`) è preferibile, perché più semplice e più veloce.

9 Grafici con `ggplot2` *(tv)*

Funzione	Scopo e uso	Attenzione
`ggplot(data, aes(...))`	Crea il grafico; le estetiche indicate qui valgono per tutti gli strati.	Gli strati si sommano con `+`, non con `%>%`.
`aes(x, y, color, fill, size, group, sample, z)`	Associa **colonne** a proprietà visive e genera la legenda.	Una costante va fuori da `aes()`: `geom_point(color="blue")`. Con `aes(color="blue")` si ottiene un livello fittizio e una legenda inutile.
`geom_point()`, `geom_line()`	Punti; linee.	`geom_line()` unisce i punti con lo stesso `group` (o `color`).
`geom_boxplot(varwidth=)`	Boxplot: mediana, quartili, baffi fino a 1.5 IQR.	La `x` deve essere categorica: se numerica, `factor()`. `fill` colora il riempimento, `color` il bordo.
`geom_histogram(bins=)`	Istogramma: numero di classi `bins` oppure larghezza `binwidth`.	Per sovrapporre una PDF usare `aes(y=after_stat(density))`, altrimenti l'ordinata è un conteggio.
`geom_function(fun=dnorm, args=list(...), xlim=)`	Disegna una funzione senza bisogno di una tabella.	Nome della funzione senza parentesi; i parametri vanno in `args`.
`geom_qq()`, `geom_qq_line()`	Diagramma Q-Q per la verifica di normalità.	Richiedono `aes(sample=...)`, non `x`/`y`.
`geom_smooth(method="lm", formula=y ~ poly(x, 2))`	Stima e disegna una regressione.	Solo grafico: per previsioni e residui serve `lm()`. Nella formula `x` e `y` sono le **estetiche**, non i nomi delle colonne.
`geom_ribbon(aes(ymin=, ymax=), alpha=)`	Banda tra due curve (intervalli di confidenza o predizione).	
`geom_vline(xintercept=)`, `geom_hline(yintercept=)`	Linee di riferimento verticali e orizzontali.	
`geom_rug()`	Tacche sull'asse in corrispondenza delle osservazioni.	
`geom_contour_filled()`, `geom_contour()`	Curve di livello di un campo scalare, con l'estetica `aes(z=...)`.	Richiedono una griglia regolare (`expand.grid()`).
`data=` in un `geom_*()`	Usa una tabella diversa per un solo strato.	
`labs()`, `theme_bw()`, `gray(l)`	Titoli ed etichette; tema grafico; grigio da 0 (nero) a 1 (bianco).	In `labs()` il titolo della legenda si dà col nome dell'estetica, es. `color=`.

Funzione	Scopo e uso	Attenzione
`facet_wrap(~g, nrow=)`	Un sottografico per ogni livello di `g`.	
`coord_cartesian(ylim=)`	Limita l'area visualizzata.	Non elimina dati: curve e modelli restano calcolati su tutti i punti.

10 Modelli

10.1 Sintassi delle formule

Formula	Significato
`y ~ x`	y in funzione di x; l'intercetta è implicita
`y ~ x - 1`	modello senza intercetta
`y ~ x + I(x^2)`	termine quadratico: senza `I()`, `x^2` si riduce a `x`
`y ~ poly(x, 2)`	polinomio **ortogonale** di grado 2; `raw=TRUE` per quello **grezzo**
`y ~ a + b`	effetti principali di due fattori
`y ~ a:b`	solo l'interazione
`y ~ a * b`	equivale a `a + b + a:b`
`y ~ factor(x)`	tratta `x` come variabile categorica
`~ g`	mezza formula: `facet_wrap(~g)`, `ggTukey(..., splt=~g)`

Polinomi ortogonali o grezzi?

Per scegliere il grado si stima un modello sovradimensionato con `poly(x, n)` (ortogonale): le stime non sono correlate, quindi la significatività di un termine non dipende dagli altri termini presenti. Se poi servono i valori fisici dei coefficienti, si stima il modello ridotto con `raw=TRUE`. I due modelli dello stesso grado danno le stesse previsioni.

10.2 Stima, verifica e previsione

Funzione	Scopo e uso	Attenzione
`lm(formula, data)`	Modello lineare nei parametri (regressione e ANOVA).	Un fattore con livelli numerici va convertito con `factor()`. Controllo: in `anova()` i `Df` del fattore devono essere pari a livelli $-$ 1.
`summary(m)`	Stime, errori standard, p-value (H_0: coefficiente nullo), R^2.	Con i polinomi grezzi la multicollinearità gonfia gli errori standard e i p-value.
`coef(m)`	Vettore dei coefficienti stimati.	Con `poly()` ortogonale i coefficienti non hanno significato fisico.
`anova(m)`	Tabella ANOVA.	Il risultato è valido solo se i residui sono normali.
`aov(formula, data)`	Come `lm()`, ma specifico per ANOVA.	È l'oggetto richiesto da `TukeyHSD()`.

Funzione	Scopo e uso	Attenzione
`TukeyHSD(aov, conf.level=)`	Confronta a coppie le medie dei trattamenti.	Vuole un oggetto `aov`, non `lm`. Una coppia è significativa se il suo intervallo non contiene lo 0.
`ggTukey()` *(adas)*	Grafico di Tukey: `TukeyHSD(...) %>% ggTukey()` oppure `df %>% ggTukey(y ~ a, splt=~b)`.	Con due fattori, `splt` produce un grafico per ogni livello del secondo.
`residuals(m)`, `fitted(m)`	Residui; valori previsti sui dati usati per la stima.	
`predict(m, newdata=)`	Valuta il modello su nuovi valori dei predittori.	`newdata` deve contenere **tutti** i predittori, con gli stessi nomi di colonna. Fuori dall'intervallo dei dati si estrapola.
`add_predictions(m)`, `add_residuals(m)` *(adas)*	Aggiungono a una tabella le colonne `pred` e `resid` (dentro una pipe).	Con `interval="confidence"` o `"prediction"` le colonne diventano `fit`, `lwr`, `upr`. L'intervallo di predizione è sempre più largo di quello di confidenza.
`glm(formula, family="binomial", data)`	Regressione logistica per una risposta binaria (`TRUE`/`FALSE`).	Le previsioni sono probabilità solo con `type="response"`, altrimenti restano in scala logit.
`nls(y ~ f(x, a, b), data, start=list(a=, b=))`	Minimi quadrati non lineari. `summary()` riporta errori standard e p-value.	Servono valori iniziali ragionevoli (rischio di mancata convergenza o di minimo locale). Non esiste R^2: verificare con un grafico e con i residui.
`optim(par, fn, ...)`	Minimizzazione numerica di una funzione obiettivo.	`fn` riceve il vettore dei parametri come **primo** argomento. Risultato in `$par`; `$convergence == 0` indica successo.
`AIC(m)`	Confronto tra modelli, anche non lineari: più basso è migliore.	Confrontare solo modelli stimati sugli stessi dati.

11 Errori più frequenti

1. `NA` nei dati: le statistiche restituiscono `NA` → `na.rm=TRUE` oppure `na.omit()`.
2. Fattore codificato con numeri: ANOVA e boxplot sbagliati senza segnalare errori → `factor()` e verifica dei `Df`.
3. `x^2` in una formula senza `I()`: il termine quadratico sparisce.
4. `%>%` al posto di `+` tra gli strati di `ggplot`, o costanti dentro `aes()`.
5. `t.test()` esegue Welch per default. Calcolando a mano un p-value bilaterale con `pt()` serve il fattore 2.
6. `if` dentro `mutate()` → `ifelse()`, `cut()` o `map_*()`.
7. Previsioni di `glm()` senza `type="response"`.
8. Gruppi rimasti attivi dopo `group_by()` o `rowwise()` → `.groups="drop"`, `.by=`, `ungroup()`.

9. Risultati casuali non riproducibili → `set.seed()`.